



Name: Muhammad Abdullah Asim

ID: ma10020

Lab 3: Implementing Jump and Return in RISC V Assembly Language

Objectives

This lab introduces RISC-V assembly instructions related to control transfer, specifically jump and return operations. Students will implement and analyze jump-based program flow using Visual Studio Code by completing a series of hands-on tasks designed to strengthen their understanding of subroutines and control flow in RISC-V.

Introduction

In this lab we will learn about:

- RISC-V Jump and Return instructions
- Handle the stack

Function calls

Functions, or procedures as they are sometimes called, are an important part of programming. They help avoid repeating the same code all over our programs. A function call would take the program flow to the start of the function code. That code will be executed, and a return from-function instruction would take the program flow back to where originally the function was called from and continue the execution from the next instruction after the function call.

Like all instruction sets, RISC-V provides special instructions to enable function calls and return from functions.

Category	Instruction	Assembly	Operation	Description
Unconditional branch	Jump and link	<code>jal x1, 100</code>	$x1 = PC+4$; go to $PC+100$	PC-relative procedure call
	Jump and link register	<code>jalr x1, 100(x5)</code>	$x1 = PC+4$; go to $x5+100$	Procedure return; indirect call

Table 3.1: RISC-V Unconditional Branch Instructions

You can think of a procedure like a spy who leaves with a secret plan, acquires resources, performs the task,



covers his or her tracks, and then returns to the point of origin with the desired result. Nothing else should be perturbed once the mission is complete. Moreover, a spy operates on only a “need to know” basis, so the spy can’t make assumptions about the spymaster. Similarly, in the execution of a procedure, the program must follow these six steps:

- Put parameters in a place where the procedure can access them.
- Transfer control to the procedure.

- Acquire the storage resources needed for the procedure.
- Perform the desired task.
- Put the result value in a place where the calling program can access it.
- Return control to the point of origin, since a procedure can be called from several points in a program

```
jal  x1, func_name
jalr x0, 0(x1)
```

Listing 1: Function instructions

Listing 1 lists the jal and jalr instructions used for function calls and returns from functions respectively in RISC-V.

Executing jal instruction would make the program flow jump to func name label in the code while at the same time storing the address of the next instruction PC+4 in the register x1. The return address needs to be stored so that the program flow can go back to where the function call was made to continue execution from there onwards once it has finished executing the function code. Since a function can be called from multiple locations in a program, without saving the return address we will not have a clue as to where to return to. The jalr instruction is used to return from functions. The jump target address is provided in a register. The example shown in Listing 1, it will jump to the address stored in the register x1. Combining this with jal instruction we can see how the two instructions work in tandem to provide the functionality of calling a function and returning from it.

RISC-V Registers			
Register	ABI Name	Description	Saved by
x0	zero	zero register (hardwired zero)	
x1	ra	return address	Caller
x2	sp	stack pointer	Callee
x3	gp	global pointer	
x4	tp	thread pointer	
x5–7	t0–2	temporary registers	Caller
x8	s0 / fp	saved register / frame pointer	Callee
x9	s1	saved register	Callee
x10–11	a0–1	function arguments / return values	Caller
x12–17	a2–7	function arguments	Caller
x18–27	s2–11	saved registers	Callee
x28–31	t3–6	temporary registers	Caller

Table 3.2: RISC-V Registers

Another issue associated with function calls is passing the parameters and returning the result. As can be seen from Table 3.2, the RISC-V convention reserves the registers x10-x17 for passing parameters to a function when calling it. The caller code would fill in these registers the parameters needed by a function before calling it. Once the function is called, it will access its parameters from these registers, use them, and then return the result in x10 before calling the jalr instruction which would take the control flow to the caller who can then access the result of the function call by reading x10.

In case the registers x10-x17 contains some values, which will still be used by the caller code after the function call, the caller code needs to store these values as well as its own return address x1 on the stack before overwriting these values with the ones needed by the callee. See section 2.8 in the textbook.

Environment Calls

The Venus simulator supports some system calls, which it calls Environment Calls, as well. See the url <https://github.com/kvakil/venus/wiki/Environmental-Calls>. To execute an environment call we have to put its number in the register a0(x10), any argument it might need in a1(x11) and then call the ecall instruction. The simulator will perform the system call specified in the a0(x10) register.

The RISC-V environment calls can be used to print values from registers or memory in the Venus Terminal. For example, one of the supported system calls is print int whose number is 1 and which will print any value stored in the register a1(x11) in the terminal as an integer.

```
addi a0 x0 1      # print int  ecall  
addi a1 x0 42     # integer 42  
ecall
```

Listing 2: Print integer

The code in listing 2, taken from the above url, would print the integer 42 on the Venus Terminal Window.

Task 1:

Write a code for the function sum shown in listing 3. It takes two integers a and b as arguments and returns their sum. You should also write code that calls this function i.e. it sets up the two arguments a and b in registers x10 and x11 and then calls sum(). After calling sum(), it then retrieves the returned result from the register x10. It can display the result using the ecall instruction.

```
int sum(int a, int b)
{
    return a+b;
}
```

```
addi x10,x0,12
addi x11,x0,12
jal x1,sum #calls the function named sum
addi x11,x10,0
li x10,1
ecall
j exit
sum:
    add x10,x11,x10
    jalr x0,0(x1)
exit:
```

Listing 3: Sum

Code

```
ASM task1.s
1  .text
2  .globl main
3
4
5  main:
6      addi x10, x0, 12
7      addi x11, x0, 12
8      jal x1, sum
9
10     addi x11, x10, 0
11     li x10, 1
12     ecall
13
14     j exit
15  sum:
16     add x10, x11, x10
17     jalr x1, 0(x1)
18
19  exit:
```



Output

```
Starting program C:\Users\HP\Desktop\4th Sem\CA\Lab 3\task1.s
```

```
24
```

```
Stop program execution!
```

Integer

```
x00 (zero) = 0x00000000
x01 (ra)    = 0x00000024
x02 (sp)    = 0x7FFFFFF0
x03 (gp)    = 0x10000000
x04 (tp)    = 0x00000000
x05 (t0)    = 0x00000000
x06 (t1)    = 0x00000000
x07 (t2)    = 0x00000000
x08 (s0)    = 0x00000000
x09 (s1)    = 0x00000000
x10 (a0)    = 0x00000001
x11 (a1)    = 0x00000018
x12 (a2)    = 0x00000000
x13 (a3)    = 0x00000000
x14 (a4)    = 0x00000000
x15 (a5)    = 0x00000000
x16 (a6)    = 0x00000000
x17 (a7)    = 0x00000000
```

Task 2:

Have a look at a C procedure:

```
long long int leaf_example (long long int g, long long int h, long long int i,  
long long int j)  
{  
    long long int f;  
    f = (g + h) - (i + j);  
    return f;  
}
```

What is the RISC-V assembly code for the above procedure?

The parameter variables g, h, i, and j correspond to the argument registers x10, x11, x12, and x13 and f corresponds to register x20. In addition, the program uses two temporary registers x18 and x19. Thus, we need to save three registers x18, x19, and x20 onto the stack.

Listing 4: Leaf Procedure

Code



```
ASM task2.s
1  .text
2  .globl main
3
4
5  main:
6      addi x10, x0, 10 #g
7      addi x11, x0, 5  #h
8      addi x12, x0, 7  #i
9      addi x13, x0, 2  #j
10     jal x1, leaf
11
12     li x10, 1
13     ecall
14     j exit
15
16 leaf:
17     add x18, x10, x11
18     add x19, x12, x13
19     sub x20, x18, x19
20     addi sp, sp, -12
21     sw x18, 8(sp)
22     sw x19, 4(sp)
23     sw x20, 0(sp)
24
25     addi x11, x20, 0
26     lw x20, 0(sp)
27     lw x19, 4(sp)
28     lw x18, 8(sp)
29     addi sp, sp, 12
30     jalr x0, 0(x1)
31 exit:
```



Output

```
-----  
Starting program C:\Users\HP\Desktop\4th Sem\CA\Lab 3\task2.s
```

```
6
```

```
x05 (t0) = 0x00000000  
x06 (t1) = 0x00000000  
x07 (t2) = 0x00000000  
x08 (s0) = 0x00000000  
x09 (s1) = 0x00000000  
x10 (a0) = 0x00000001  
x11 (a1) = 0x00000006  
x12 (a2) = 0x00000007  
x13 (a3) = 0x00000002  
x14 (a4) = 0x00000000  
x15 (a5) = 0x00000000  
x16 (a6) = 0x00000000  
x17 (a7) = 0x00000000  
x18 (s2) = 0x0000000F  
x19 (s3) = 0x00000009  
x20 (s4) = 0x00000006  
x21 (s5) = 0x00000000  
x22 (s6) = 0x00000000  
x23 (s7) = 0x00000000  
x24 (s8) = 0x00000000  
x25 (s9) = 0x00000000  
x26 (t3) = 0x00000000
```

```
WATCH
```



Task 3:

The following C procedure swaps two array elements (or two locations in memory)

```
void swap (long long int v[ ], size_t k)
{
    long long int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

What is the RISC-V assembly code for the above procedure?

The parameter variables $v[]$ and k correspond to the argument registers $x10$ and $x11$.

Listing 5: Swap Procedure

Code



```
ASM task1.s    ASM task2.s    ASM task3.s ●
ASM task3.s
1  .text
2  .globl main
3
4  main:
5      li x10, 0x100
6      li x5, 2
7      sw x5, 0(x10)
8      li x5, 4
9      sw x5, 4(x10)
10     li x5, 8
11     sw x5, 8(x10)
12     li x5, 6
13     sw x5, 12(x10)
14     li x5, 10
15     sw x5, 16(x10)
16     li x5, 12
17     sw x5, 20(x10)
18
19     li x11, 2
20     jal x1, swap
21     addi x17, x0, 10
22     ecall
23
24  swap:
25     slli x5, x11, 2
26     add x5, x10, x5
27
28     lw x6, 0(x5)
29     lw x7, 4(x5)
30
31     sw x7, 0(x5)
32     sw x6, 4(x5)
33
34     jalr x0, 0(x1)
35
36  exit:
37     j exit
```

Output



Memory

Address	+0	+1	+2	+3
0x00000118	0	0	0	0
0x00000114	12	0	0	0
0x00000110	10	0	0	0
0x0000010C	6	0	0	0
0x00000108	8	0	0	0
0x00000104	4	0	0	0
0x00000100	2	0	0	0
0x000000FC	0	0	0	0
0x000000F8	0	0	0	0
0x000000F4	0	0	0	0
0x000000F0	0	0	0	0
0x000000EC	0	0	0	0
0x000000E8	0	0	0	0

Address: Up Down

Jump to: -- choose --

Integer

- x00 (zero) = 0x00000000
- x01 (ra) = 0x0000003C
- x02 (sp) = 0x7FFFFFF0
- x03 (gp) = 0x10000000
- x04 (tp) = 0x00000000
- x05 (t0) = 0x00000108
- x06 (t1) = 0x00000008
- x07 (t2) = 0x00000008
- x08 (s0) = 0x00000000
- x09 (s1) = 0x00000000
- x10 (a0) = 0x00000100
- x11 (a1) = 0x00000002
- x12 (a2) = 0x00000000
- x13 (a3) = 0x00000000
- x14 (a4) = 0x00000000
- x15 (a5) = 0x00000000
- x16 (a6) = 0x00000000
- x17 (a7) = 0x0000000A
- x18 (s2) = 0x00000000
- x19 (s3) = 0x00000000
- x20 (s4) = 0x00000000

WATCH

Task 4:

The procedure strcpy copies string y to string x using the null byte termination convention of C:

```
void strcpy (char x [], char y[])  
  
{  
  
    size_t i;  
  
    i = 0;  
  
    while ((x[i] = y[i]) != '\0') /* copy & test byte */  
  
        i += 1;  
  
}
```

What is the RISC-V assembly code?

Code

```
ASM task4.s
1  .text
2  .globl main
3
4  main:
5      li x5, 0x00616554    # "tea"
6      li x6, 0x200
7      sw x5, 0(x6)
8      li x10, 0x100
9      li x11, 0x200
10     jal x1, strcpy
11
12     exit:
13     j exit
14
15     strcpy:
16         addi sp, sp, -8
17         sw x19, 0(sp)
18         addi x19, x0, 0
19
20     loop_start:
21         add x5, x19, x11
22         lbu x6, 0(x5)
23
24         add x7, x19, x10
25         sb x6, 0(x7)
26
27         beq x6, x0, end
28
29         addi x19, x19, 1
30         jal x0, loop_start
31
32     end:
33         lw x19, 0(sp)
34         addi sp, sp, 8
35         jalr x0, 0(x1)
```

Output



Integer

x00 (zero) = 0x00000000
x01 (ra) = 0x0000001C
x02 (sp) = 0x7FFFFFFF
x03 (gp) = 0x10000000
x04 (tp) = 0x00000000
x05 (t0) = 0x00000203
x06 (t1) = 0x00000000
x07 (t2) = 0x00000103
x08 (s0) = 0x00000000
x09 (s1) = 0x00000000
x10 (a0) = 0x00000100
x11 (a1) = 0x00000200
x12 (a2) = 0x00000000
x13 (a3) = 0x00000000
x14 (a4) = 0x00000000
x15 (a5) = 0x00000000
x16 (a6) = 0x00000000
x17 (a7) = 0x00000000

Address	+0	+1	+2	+3
0x00000218	00	00	00	00
0x00000214	00	00	00	00
0x00000210	00	00	00	00
0x0000020C	00	00	00	00
0x00000208	00	00	00	00
0x00000204	00	00	00	00
0x00000200	54	65	61	00
0x000001FC	00	00	00	00
0x000001F8	00	00	00	00
0x000001F4	00	00	00	00
0x000001F0	00	00	00	00
0x000001EC	00	00	00	00
0x000001E8	00	00	00	00

Address: Up Down

Jump to: -- choose --

Display Format: Hex

Bytes per Row: 4



Assessment Rubric

Lab 3: Implementing Jump and Return Introductions in RISC V Assembly Language

Name: Muhammad Abdullah Asim	Student ID: ma10020	Section: T3
-------------------------------------	----------------------------	--------------------

Points Distribution

	Task No.	LR 2 Code	LR 5 Results
In - Lab	Task 1	/10	/05
	Task 2	/10	/10
	Task 3	/10	/10
	Task 4	/15	/10
Total Points: 100		/45	/35
CLO Mapped		CLO 2	

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission & Git Upload	/10 & /10	CLO 2

CLO	Total Points	Points Obtained
2	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.